

Lab 2 on Regression:

Introduction to Logistic Regression

Dataset: Titanic Passenger Survival

Goal: Predict if a passenger survived (1) or died (0) based on their age, ticket fare, and gender.

1. The Theory: What is Logistic Regression?

While Linear Regression draws a straight line to predict a continuous number (like price or sales), **Logistic Regression** is used for **Classification**. It predicts the *probability* that an event will happen (between 0 and 1) using an S-shaped curve called the Sigmoid curve.

- **The Math:**

$$P(Y = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X)}}$$

- **Rule of Thumb:** If the predicted probability is > 0.50 , the model classifies the outcome as 1 (Yes). If it is < 0.50 , it classifies it as 0 (No).

2. Environment Setup & Data Loading

Run the following code to load the dataset and do some basic cleaning so the math doesn't break on missing values.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Load the built-in Titanic dataset
df = sns.load_dataset('titanic')

# 2. Keep only the columns we need for this lab
df = df[['survived', 'age', 'fare', 'sex']]

# 3. Drop rows with missing ages
df = df.dropna()

# 4. Machine Learning requires numbers, not text.
# Convert 'sex' (male/female) into a binary column: 'is_female' (0 or 1)
df['is_female'] = (df['sex'] == 'female').astype(int)

print(df.head())
```

3. Your Tasks

Task 1: Visualizing the S-Curve

Using `sns.regplot()`, plot age (X-axis) against survived (Y-axis).

- *Hint:* To make Seaborn draw a logistic S-curve instead of a straight linear line, you must add the parameter `logistic=True` to your function. (Note: this might take a few seconds to calculate!).

Task 2: Train the Logistic Regression Model

We will use the scikit-learn library to build our model.

1. Define your predictors (X): Use age, fare, and is_female.
2. Define your target (y): Use survived.
3. Import LogisticRegression from `sklearn.linear_model`.
4. Fit the model using `model.fit(X, y)`.

Task 3: Model Evaluation (The Confusion Matrix)

A model is only as good as its predictions. We evaluate classification models using a "Confusion Matrix" to see exactly where the model got it right and where it got it wrong.

1. Use your model to generate predictions on your X data: `predictions = model.predict(X)`
2. Import `confusion_matrix` and `ConfusionMatrixDisplay` from `sklearn.metrics`.
3. Plot the confusion matrix comparing your actual y values to your predictions.
4. **Answer this question in a markdown cell:** Look at your matrix. How many people did the model *correctly* predict as survivors?

Task 4 : The "Jack and Rose" Test (Custom Predictions)

So far, you have evaluated how the model performed on historical data. Now, let's use your trained model to predict the future.

While `model.predict()` just gives you a 1 (Survived) or 0 (Died), Logistic Regression models can also give you the exact mathematical percentage chance of an outcome using `model.predict_proba()`.

Your Task: Create two fictional passengers and calculate their exact percentage chance of survival.

1. **Passenger A ("Jack"):** 20 years old, paid a \$8 fare, is male (`is_female = 0`).
2. **Passenger B ("Rose"):** 22 years old, paid a \$150 fare, is female (`is_female = 1`).
3. Create a new DataFrame with these values. Ensure the columns are in the exact same order as the data you trained the model on (age, fare, `is_female`).

4. Use `model.predict_proba()` on this new DataFrame to find their survival probabilities.
5. **Markdown Answer:** What is the exact percentage chance of survival for each passenger?